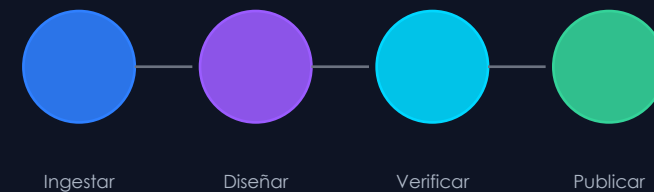


Microservice Code Studio Arquitectura & Java 21 Autónomo

De especificaciones formales BDD a microservicios Java 21 /
Spring Boot 3 con sandbox hermético, auto-reparación y
control de calidad.

RETO TÉCNICO

Negocio + IA + Evidencia



La propuesta no busca solo 'escribir código'.
Construye microservicios Java 21
herméticamente probados, alineados a la
Constitución y listos para producción.

Dolor y oportunidad

Desarrollo manual repetitivo (días por servicio): Escribir boilerplate de controladores, servicios, repositorios, entidades JPA y DTOs consume 70% del tiempo de sprint.

Código alucinado y sin verificación real: LLMs convencionales generan dependencias incompatibles, fallos de tipado y código que no compila sin un entorno de pruebas hermético.

Brecha entre requerimientos BDD y código Java: Desconexión entre criterios Given/When/Then, contratos REST OpenAPI, esquemas SQL y pruebas unitarias Mockito.

Ausencia de gobierno, SAST y despliegue: Riesgo de secretos persistidos en disco/BD, sin compuertas de calidad (Quality Gate) ni contenedores Docker/K8s listos para producción.

Tesis para el jurado: No generamos código por generar; construimos un estudio autónomo con Spec-Driven Development (SDD), compilación hermética (--network none) y auto-reparación acotada para garantizar microservicios Java 21 certificados.

De la idea al impacto

Mantenemos el esquema del reto: cuatro fases centradas en valor, usuario y evidencia.



Principios que elevaron la calidad

Usuario

Datos

IA responsable

Evidencia

IA guiada por especificaciones

El valor no vino de una sola herramienta: vino de orquestar personas, specs, modelos y validación hermética.

Spec Driven Development

- Specify / Spec Kit: Formalización de la Constitución v1.1.0, historias de usuario BDD y especificaciones ejecutables (spec.md, plan.md, tasks.md).
- LangGraph Orchestration: Grafo de estados con nodos especializados (Scaffolding, Modelos JPA, DTOs Records, Servicios, Controllers y Tests Mockito).
- Hermetic Docker Sandbox: Aislamiento estricto con --network none y compilación offline (mvn test -o) contra repositorio Maven local de solo lectura.
- Self-Repair & Pytest Suite: Máquina de auto-reparación acotada (máx 3 intentos) + suite de 125 tests automatizados pasando al 100%.

Modelo y runtime usados

Google Gemini 2.5 Flash / Groq · Python 3.12 · FastAPI · LangGraph · Streamlit · Docker · Java 21 LTS · Spring Boot 3.2 · Maven · PostgreSQL / SQLite

Microservice Code Studio

- **Inicio Rápido & Auto-Pilot:** Síntesis autónoma 1-Click o modo asistido con controles hot-pause, resume y cancel en caliente.
- **Ingesta Dual BDD (Spec Kit):** Carga de spec.md o JSON blueprint con validación formal de criterios Given/When/Then.
- **Arquitectura 4 Capas & JPA:** Topología limpia (Controller→Service→Repo→Model), DTOs Java Records inmutables y scripts SQL sincronizados.
- **Sandbox Hermético Offline:** Compilación y pruebas Mockito con --network none contra caché .m2 de solo lectura inmutable.
- **Auto-Reparación Acotada (3x):** Diagnóstico automático de trazas Maven con auto-parcheo de código antes de escalar a HITL.
- **Quality Gate SAST & DevOps:** Cero secretos persistidos, reporte estático 100/100, Dockerfile multi-stage, Compose y K8s.

Ingesta BDD &
Auto-Pilot



Sandbox Docker
Hermético



Auto-
Reparación
Acotada (3x)

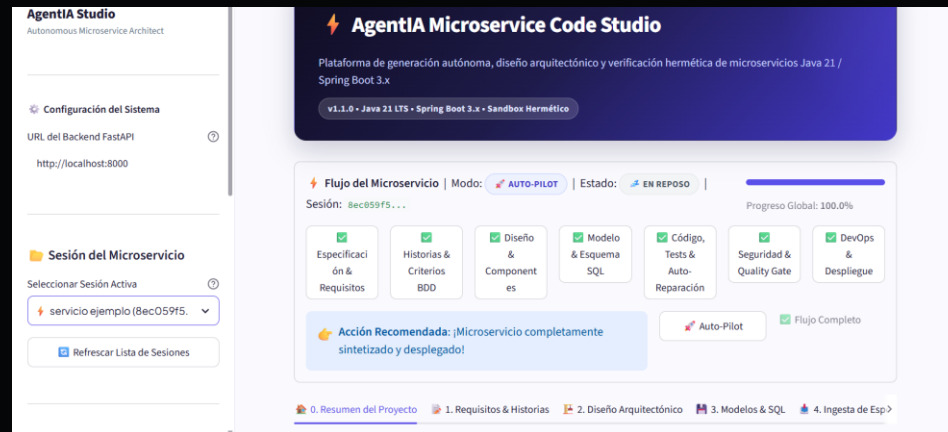


Quality Gate &
DevOps K8s

CIERRE GANADOR

Evidencia, impacto y escalabilidad

Una solución técnica real convertida en activo estratégico para la aceleración del ciclo de software.



Highlights

Impacto de negocio

Reduce el tiempo de creación de microservicios de semanas a menos de 2 minutos, elimina el 100% de errores de boilerplate y estandariza arquitectura Java 21 sin deuda técnica.

Solidez técnica

No es un generador alucinado: cuenta con sandbox Docker hermético (--network none), 125 tests automatizados al 100%, compilación Maven real y auto-reparación acotada.

Escala responsable

Cumplimiento de la Constitución v1.1.0: cero secretos persistidos, compuertas de calidad (Quality Gate), auditoría SAST con auto-parcheo y control humano innegociable (HITL).

Siguiente paso ejecutivo: Desplegar piloto en equipos de desarrollo corporativo, integrar con el catálogo central de APIs y habilitar plantillas de microservicios cloud-native.